

FIFO

SV VERIFICATION ENVIRONMENT



Table of Contents

1-Introduction	3
2- FIFO Design & Functionality Overview.....	4
2.1-Main Signals.....	6
2.2- FIFO Operation.....	7
2.3 Verification Environment Architecture	8
3- Verification Methodology.....	9
3.1 Verification Plan.....	9
3.2 Codes.....	12
4- Simulation Results & Coverage.....	26
4.1 Waveform.....	26
4.2 Code Coverage Report.....	27
4.3 Functional Coverage Report.....	32
4.4 Assertions Report	35

1- Introduction

First-In First-Out (FIFO) memories are widely used in digital systems to buffer data between functional blocks and to manage differences in data production and consumption rates. A synchronous FIFO operates using a single clock domain and provides controlled read and write access through status flags such as full, empty, almost full, and almost empty, ensuring safe and reliable data transfer.

The objective of this project is to **verify the functionality and correctness of a parameterized synchronous FIFO design** using SystemVerilog. The FIFO supports configurable data width and depth, and provides additional control and status signals including write acknowledge, overflow, and underflow indicators. Due to the critical role of FIFOs in digital designs, thorough verification is required to ensure correct behavior under all operating conditions.

A simulation-based verification environment was developed to validate the FIFO design. The verification approach combines **constrained-random stimulus generation, a reference model-based scoreboard, functional coverage, and SystemVerilog assertions**. This methodology ensures that all FIFO states and corner cases—such as full, empty, reset conditions, and simultaneous read/write operations—are properly exercised and checked.

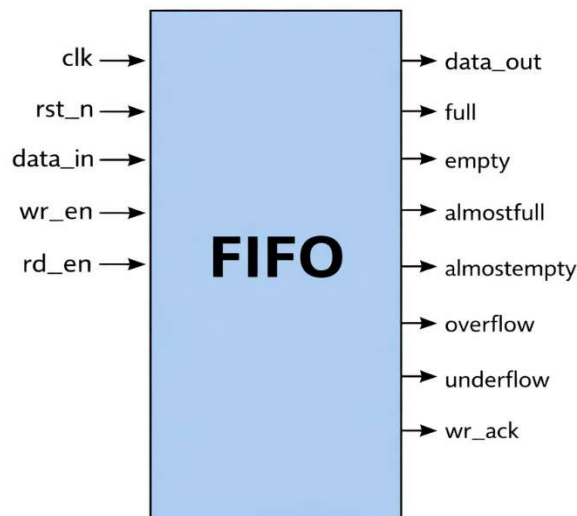
The goal of this verification effort is to achieve complete functional correctness and full coverage, as well as to identify and fix any design bugs detected during simulation.

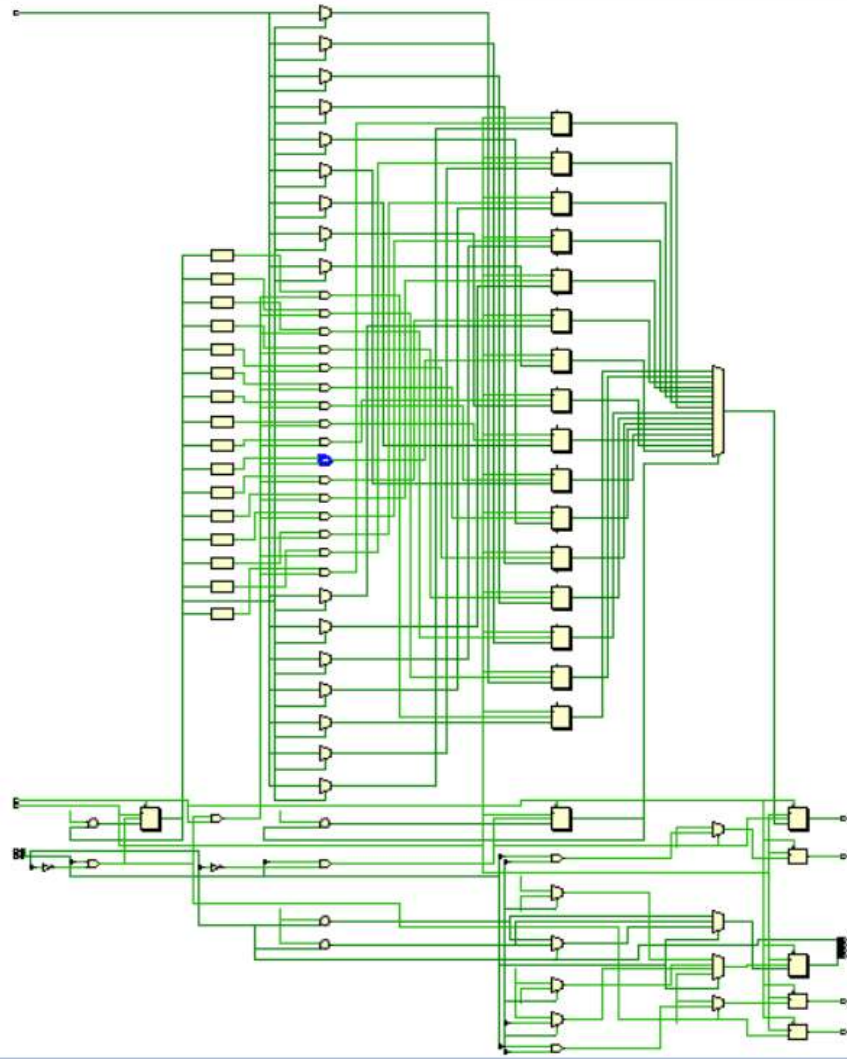
2- FIFO Design & Functionality Overview

The implemented FIFO consists of a memory array, read and write pointers, and a counter that tracks the number of stored elements. The write pointer advances when valid write operations occur, while the read pointer advances during valid read operations. The internal counter updates accordingly to reflect the current occupancy of the memory.

Control logic generates status signals based on the counter value. When the counter reaches zero, the FIFO is considered empty, and when it reaches the configured depth, the FIFO is full. Intermediate thresholds are used to assert the almost full and almost empty indicators. Overflow and underflow signals are produced when invalid write or read attempts occur, ensuring that the memory contents remain protected.

All operations are synchronized with the system clock, and an active-low asynchronous reset initializes the internal state by clearing pointers and counters.





2.1 Main Signals

Signal Name	Direction	Description
clk	Input	System clock. All read and write operations occur on the clock edge.
rst_n	Input	Active-low asynchronous reset. When asserted, all internal pointers and counters are reset and the FIFO becomes empty.
data_in	Input	Input data bus used to write data into the FIFO.
wr_en	Input	Write enable signal. When asserted and the FIFO is not full, data is written into the FIFO.
rd_en	Input	Read enable signal. When asserted and the FIFO is not empty, data is read from the FIFO.
data_out	Output	Output data bus that provides the read data from the FIFO.
full	Output	Indicates that the FIFO has reached its maximum capacity and cannot accept new writes.
empty	Output	Indicates that the FIFO contains no valid data.
almostfull	Output	Asserted when the FIFO is one write operation away from being full.
almostempty	Output	Asserted when the FIFO is one read operation away from being empty.
overflow	Output	Asserted when a write is attempted while the FIFO is full.
underflow	Output	Asserted when a read is attempted while the FIFO is empty.
wr_ack	Output	Indicates that a write operation has been successfully completed.

These signals collectively ensure controlled data transfer and provide real-time status feedback about the internal state of the FIFO, enabling safe and reliable operation.

2.2 FIFO Operation

Write Operation:

- Write occurs when `wr_en` = 1 and FIFO is **not full**
- `wr_ack` is asserted on a successful write
- If FIFO is full, write is ignored and overflow is asserted

Read Operation

Read occurs when `rd_en` = 1 and FIFO is **not empty**

Output data appears on `data_out`

If FIFO is empty, read is ignored and underflow is asserted

Simultaneous Read & Write

- When both `wr_en` and `rd_en` are high:
 - If FIFO is empty → only write happens
 - If FIFO is full → only read happens

Reset Behavior

- Active-low asynchronous reset
- All pointers and counters reset
- FIFO goes to empty state

2.3 Verification Environment Architecture

The verification environment is built using a modular SystemVerilog testbench structure.

A top module generates the clock and reset signals and instantiates the FIFO DUT.

A SystemVerilog interface is used to connect the DUT with the testbench components.

The testbench drives constrained-random read and write transactions to stimulate different FIFO scenarios.

A monitor samples the interface signals on every clock cycle and converts them into transaction objects.

The transactions are forwarded to a scoreboard that compares DUT outputs with expected reference results.

A functional coverage component collects coverage information to ensure all FIFO states and control combinations are exercised.

The simulation ends when the test is completed, and a summary of correctness and coverage results is displayed.

3-Verification Methodology

3.1 Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	Verify that asynchronous reset clears wr_ptr, rd_ptr, and count to zero.	rst_n is randomized during simulation to trigger async resets.	Cover that reset occurred (rst_n==0) and signals went to default	reset_prop ensures pointers and counter are reset to 0.
FIFO_2	Verify that when wr_en is high and FIFO is not full, a write acknowledge (wr_ack) occurs in the next cycle.	wr_en and full conditions are randomized during simulation.	Cover all legal writes (wr_en=1, full=0).	wr_ack_prop checks wr_ack is asserted after valid write.
FIFO_3	Verify that when wr_en is high and FIFO is full, overflow flag asserts.	Randomized write operations including overflow scenarios.	Cover overflow attempts (wr_en=1, full=1).	overflow_prop ensures overflow triggers correctly.
FIFO_4	Verify that when rd_en is high and FIFO is empty, underflow flag asserts.	Randomized read operations including underflow scenarios	Cover underflow attempts (rd_en=1, empty=1).	underflow_prop checks proper underflow behavior.
FIFO_5	Ensure empty flag reflects FIFO count of zero.	Randomized read/write operations until FIFO empties.	Cover cases where count=0.	empty_flag_prop ensures empty==1 when count=0.

FIFO_6	Ensure full flag reflects FIFO count equals depth.	Randomized writes until FIFO becomes full.	Cover cases where count==FIFO_DEPTH.	full_flag_prop ensures full==1 when FIFO is full.
FIFO_7	Verify almost full flag asserts when FIFO has one space left.	Randomized write patterns driving FIFO to almost full.	Cover cases where count==DEPTH-1.	almostfull_prop checks proper almost-full indication.
FIFO_8	Verify almost empty flag asserts when FIFO has one element left.	Randomized read/write operations approaching almost-empty condition.	Cover cases where count==1.	almostempty_prop ensures almost-empty flag correctness.
FIFO_9	Verify write pointer wrap-around from last index to 0.	Randomized write sequences ensuring pointer wraps around.	Cover wrap event (wr_ptr==DEPTH-1 → wr_ptr==0).	wr_ptr_wraparound_prop ensures proper wrap around.
FIFO_10	Verify read pointer wrap around from last index to 0.	Randomized read sequences ensuring pointer wraps around.	Cover wrap event (rd_ptr==DEPTH-1 → rd_ptr==0).	rd_ptr_wraparound_prop ensures correct wrap around.
FIFO_11	Ensure write pointer never exceeds depth.	Randomized read/write activity to stress write logic.	Cover all wr_ptr values (0 → DEPTH-1).	wr_ptr_threshold_prop asserts wr_ptr < DEPTH.
FIFO_12	Ensure read pointer never exceeds depth.	Randomized read/write activity to stress read logic.	Cover all rd_ptr values (0 → DEPTH-1).	rd_ptr_threshold_prop asserts rd_ptr < DEPTH.
FIFO_13	Ensure count never exceeds FIFO depth.	Randomized read/write activity across simulation duration.	Cover all count values from 0 → DEPTH.	count_threshold_prop ensures count ≤ DEPTH.

FIFO_14	Verify asynchronous reset stimulus is randomly applied during simulation.	rst_n randomized via constraint (10% low, 90% high).	Cover multiple reset assertion/deassertion events.	Observed indirectly through reset_prop and scoreboard.
FIFO_15	Control write enable distribution to maintain realistic traffic.	wr_en randomized (WR_EN_ON_DIST weight applied dynamically).	Cover variations of write enable activation frequency.	Verified by coverage and scoreboard correctness.
FIFO_16	Control read enable distribution to maintain realistic traffic.	rd_en randomized (RD_EN_ON_DIST weight applied dynamically).	Cover variations of read enable activation frequency.	Verified by coverage and scoreboard correctness.
FIFO_17	FIFO should properly reset internal pointers, count, and flags	The assert_reset() task drives rst_n=0→1 to reset FIFO	Reset event sampled and logged	Checked in scoreboard after reset to confirm correct FIFO state
FIFO_18	FIFO should correctly handle randomized read/write operations	Randomized stimulus for wr_en, rd_en, and data_in repeated for 20,000 cycles	Covered by coverage group and transaction constraints	Checked in scoreboard for data integrity and flag correctness
FIFO_19	Verify functional interactions between write/read enables and status flags (full, empty, overflow, etc.).	Randomized read/write/flag behavior generated automatically.	Cross coverage: wr_en, rd_en, full, empty, almostfull, underflow, etc.	Verified by coverage and scoreboard correctness.
FIFO_20	Verify asynchronous reset clears all output and status flags to default values	rst_n=0 applied during active clock cycles	async_reset_cover records successful reset event	async_reset_prop assertion checks reset outputs and flags in top level module

3.2 Codes

- Desgin module & Assertions

```
module FIFO(fifo_interface.DUT fifo_if);

    localparam max_fifo_addr = $clog2(fifo_if.FIFO_DEPTH);

    reg [fifo_if.FIFO_WIDTH-1:0] mem [fifo_if.FIFO_DEPTH-1:0];

    reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
    reg [max_fifo_addr:0] count;

    always @(posedge fifo_if.clk or negedge fifo_if.rst_n) begin
        if (!fifo_if.rst_n) begin
            wr_ptr <= 0;
            fifo_if.wr_ack <= 0; // bug fixed
            fifo_if.overflow <= 0; // bug fixed
        end
        else if (fifo_if.wr_en && count < fifo_if.FIFO_DEPTH) begin
            mem[wr_ptr] <= fifo_if.data_in;
            fifo_if.wr_ack <= 1;
            wr_ptr <= wr_ptr + 1;
        end
        else begin
            fifo_if.wr_ack <= 0;
            if (fifo_if.full & fifo_if.wr_en)
                fifo_if.overflow <= 1;
            else
                fifo_if.overflow <= 0;
        end
    end

    always @(posedge fifo_if.clk or negedge fifo_if.rst_n) begin
        if (!fifo_if.rst_n) begin
            rd_ptr <= 0;
            fifo_if.underflow <= 0; // bug fixed
            fifo_if.data_out <= 0; // bug fixed
        end
        else if (fifo_if.rd_en && count != 0) begin
            fifo_if.data_out <= mem[rd_ptr];
            rd_ptr <= rd_ptr + 1;
        end
    end
end
```

```

        end
        // bug fixed
        else begin
            if (fifo_if.empty && fifo_if.rd_en)
                fifo_if.underflow <= 1;
            else
                fifo_if.underflow <= 0;
            end
        end
    end

    always @(posedge fifo_if.clk or negedge fifo_if.rst_n) begin
        if (!fifo_if.rst_n) begin
            count <= 0;
        end
        else begin
            if ( ({fifo_if.wr_en, fifo_if.rd_en} == 2'b10) && !fifo_if.full)
                count <= count + 1;
            else if ( ({fifo_if.wr_en, fifo_if.rd_en} == 2'b01) &&
!fifo_if.empty)
                count <= count - 1;
            // bug fixed
            else if ( ({fifo_if.wr_en, fifo_if.rd_en} == 2'b11) begin
                if (fifo_if.empty)
                    count <= count + 1;
                if (fifo_if.full)
                    count <= count - 1;
            end
        end
    end

    assign fifo_if.full = (count == fifo_if.FIFO_DEPTH)? 1 : 0;
    assign fifo_if.empty = (count == 0)? 1 : 0;
    assign fifo_if.almostfull = (count == fifo_if.FIFO_DEPTH-1)? 1 : 0; // bug
fixed
    assign fifo_if.almostempty = (count == 1)? 1 : 0;

`ifdef SIM
// FIFO_1
    property reset_prop;
        @(posedge fifo_if.clk)
        (!fifo_if.rst_n) |-> ##1 (wr_ptr == 0) && (rd_ptr == 0) && (count == 0);
    endproperty

//FIFO_2
    property wr_ack_prop;

```

```

        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (fifo_if.wr_en && !fifo_if.full) |-> ##1 fifo_if.wr_ack;
    endproperty

//FIFO_3
    property overflow_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (fifo_if.wr_en && fifo_if.full) |-> ##1 fifo_if.overflow;
    endproperty

//FIFO_4
    property underflow_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (fifo_if.rd_en && fifo_if.empty) |-> ##1 fifo_if.underflow;
    endproperty

//FIFO_5
    property empty_flag_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (count == 0) |-> fifo_if.empty;
    endproperty

//FIFO_6
    property full_flag_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (count == fifo_if.FIFO_DEPTH) |-> fifo_if.full;
    endproperty

//FIFO_7
    property almostfull_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (count == fifo_if.FIFO_DEPTH - 1) |-> fifo_if.almostfull;
    endproperty

//FIFO_8
    property almostempty_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (count == 1) |-> fifo_if.almostempty;
    endproperty

//FIFO_9
    property wr_ptr_wraparound_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
        (wr_ptr == fifo_if.FIFO_DEPTH - 1) && fifo_if.wr_en && (count <
fifo_if.FIFO_DEPTH)

```

```

        |-> ##1 (wr_ptr == 0);
    endproperty
//FIFO_10
    property rd_ptr_wraparound_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
            (rd_ptr == fifo_if.FIFO_DEPTH - 1) && fifo_if.rd_en && (count != 0)
            |=> (rd_ptr == 0);
    endproperty
//FIFO_11
    property wr_ptr_threshold_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
            wr_ptr < fifo_if.FIFO_DEPTH;
    endproperty
//FIFO_12
    property rd_ptr_threshold_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
            rd_ptr < fifo_if.FIFO_DEPTH;
    endproperty
//FIFO_13
    property count_threshold_prop;
        @(posedge fifo_if.clk) disable iff(!fifo_if.rst_n)
            count <= fifo_if.FIFO_DEPTH;
    endproperty

    //=====
    // Assertions and Covers
    //=====

    assert property(reset_prop);
    cover property(reset_prop);

    assert property(wr_ack_prop);
    cover property(wr_ack_prop);

    assert property(overflow_prop);
    cover property(overflow_prop);

    assert property(underflow_prop);
    cover property(underflow_prop);

    assert property(empty_flag_prop);
    cover property(empty_flag_prop);

    assert property(full_flag_prop);
    cover property(full_flag_prop);

```

```

assert property(almostfull_prop);
cover property(almostfull_prop);

assert property(almostempty_prop);
cover property(almostempty_prop);

assert property(wr_ptr_wraparound_prop);
cover property(wr_ptr_wraparound_prop);

assert property(rd_ptr_wraparound_prop)
    else $error("Read pointer wraparound failed");
cover property(rd_ptr_wraparound_prop);

assert property(wr_ptr_threshold_prop)
    else $error("Write pointer exceeded threshold");
cover property(wr_ptr_threshold_prop);

assert property(rd_ptr_threshold_prop)
    else $error("Read pointer exceeded threshold");
cover property(rd_ptr_threshold_prop);

assert property(count_threshold_prop)
    else $error("Count exceeded FIFO depth");
cover property(count_threshold_prop);

`endif

endmodule

```

- Testbench Module

```

module fifo_tb(fifo_interface.TEST fifo_if);

    import FIFO_transaction_pkg::*;
    import shared_pkg::*;
    FIFO_transaction test;

    task assert_reset();
        fifo_if.rst_n=0;
        @(negedge fifo_if.clk);
        -> check_event;
        fifo_if.rst_n=1;
    endtask

```



```

initial begin

    test = new();
    //FIFO_17
    assert_reset();
    @(negedge fifo_if.clk);
    -> check_event;
    //FIFO_18
    repeat(20000) begin
        assert(test.randomize());
        fifo_if.rst_n = test.rst_n;
        fifo_if.data_in = test.data_in;
        fifo_if.wr_en = test.wr_en;
        fifo_if.rd_en = test.rd_en;
        @(negedge fifo_if.clk);
        -> check_event;
    end

    test_finished = 1;
end
endmodule

```

- Monitor

```

import shared_pkg::*;
import FIFO_transaction_pkg::*;
import FIFO_coverage_pkg::*;
import FIFO_scoreboard_pkg::*;
module fifo_monitor (fifo_interface.MON fifo_if);

    FIFO_transaction trans;
    FIFO_coverage cov;
    FIFO_scoreboard sb;

    initial begin
        trans = new;
        cov = new;
        sb = new;
        forever begin
            wait(check_event.triggered);
            @(negedge fifo_if.clk);

```

```

trans.data_in = fifo_if.data_in;
trans.rst_n = fifo_if.rst_n;
trans.wr_en = fifo_if.wr_en;
trans.rd_en = fifo_if.rd_en;
trans.data_out = fifo_if.data_out;
trans.full = fifo_if.full;
trans.empty = fifo_if.empty;
trans.almostfull = fifo_if.almostfull;
trans.almostempty = fifo_if.almostempty;
trans.wr_ack = fifo_if.wr_ack;
trans.overflow = fifo_if.overflow;
trans.underflow = fifo_if.underflow;

fork
    begin
        cov.sample_data(trans);
    end
    begin
        sb.check_data(trans);
    end
join

if (test_finished) begin
    $display("Simulation done at time %t" , $time);
    $display("Correct Count: %0d , Error Count: %0d", correct_count,
error_count);
    $stop;
end
end
end

endmodule

```

- Coverage Class

```
package FIFO_coverage_pack;
import FIFO_transaction_pkg::* ;
import shared_pkg::*;
class FIFO_coverage;
FIFO_transaction F_cvg_txn;

covergroup CovGroup ;

//FIFO_19
write_en_cp: coverpoint F_cvg_txn.wr_en
{
    bins wr_en_on    = {1};
    bins wr_en_off   = {0};
}

read_en_cp: coverpoint F_cvg_txn.rd_en
{
    bins rd_en_on    = {1};
    bins rd_en_off   = {0};
}

full_cp: coverpoint F_cvg_txn.full
{
    bins full_high    = {1};
    bins full_low     = {0};
}

almostfull_cp: coverpoint F_cvg_txn.almostfull
{
    bins almostfull_high = {1};
    bins almostfull_low  = {0};
}

empty_cp:    coverpoint F_cvg_txn.empty
{
    bins empty_high = {1};
    bins empty_low  = {0};
}

almostempty_cp:    coverpoint F_cvg_txn.almostempty
{
    bins almostempty_high = {1};
    bins almostempty_low  = {0};
}
```

```

}

overflow_cp:    coverpoint F_cvg_txn.overflow
{
bins overflow_high = {1};
bins overflow_low  = {0};
}

underflow_cp:   coverpoint F_cvg_txn.underflow
{
bins underflow_high = {1};
bins underflow_low  = {0};
}

wr_ack_cp:      coverpoint F_cvg_txn.wr_ack
{
bins wr_ack_high = {1};
bins wr_ack_low  = {0};
}

wr_rd_full: cross write_en_cp , read_en_cp , full_cp
{
bins read_write_full = binsof(write_en_cp) && binsof(read_en_cp) &&
binsof(full_cp) ;
}

wr_rd_almostfull: cross write_en_cp , read_en_cp , almostfull_cp
{
bins read_write_almostfull = binsof(write_en_cp) && binsof(read_en_cp) &&
binsof(almostfull_cp) ;
}

wr_rd_empty: cross write_en_cp , read_en_cp , empty_cp
{
bins read_write_empty = binsof(write_en_cp) && binsof(read_en_cp) &&
binsof(empty_cp) ;
}

wr_rd_almostempty: cross write_en_cp , read_en_cp , almostempty_cp
{
bins read_write_almostempty = binsof(write_en_cp) && binsof(read_en_cp) &&
binsof(almostempty_cp) ;
}

wr_rd_overflow: cross write_en_cp , read_en_cp , overflow_cp

```

```

{
    bins read_write_overflow = binsof(write_en_cp) && binsof(read_en_cp) &&
    binsof(overflow_cp) ;
}

wr_rd_underflow : cross write_en_cp , read_en_cp , underflow_cp
{
    bins read_write_underflow = binsof(write_en_cp) && binsof(read_en_cp) &&
    binsof(underflow_cp) ;
}

wr_rd_wrack: cross write_en_cp , read_en_cp , wr_ack_cp
{
    bins read_write_ack = binsof(write_en_cp) && binsof(read_en_cp) &&
    binsof(wr_ack_cp) ;
}

endgroup

function new();
    CovGroup = new();
    F_cvg_txn = new();
endfunction

function void sample_data (FIFO_transaction F_txn);
    F_cvg_txn = F_txn;
    CovGroup.sample();
endfunction

endclass
endpackage

```

- Scoreboard

```
package FIFO_scoreboard_pkg;
import FIFO_transaction_pkg::*;
import shared_pkg::*;
logic [15:0]data_view;
class FIFO_scoreboard;
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    bit [FIFO_WIDTH-1:0] data_out_ref;
    bit [FIFO_WIDTH-1:0] fifo_ref[$];
    function void check_data(FIFO_transaction F_txn);
        reference_model(F_txn);
        if(data_out_ref != F_txn.data_out) begin
            error_count++;
            $display("wr_en = %0b, rd_en = %0b", F_txn.wr_en, F_txn.rd_en);
            $display("Error at time = %0t : Expected = %0h, Got = %0h",
                $time, data_out_ref, F_txn.data_out);
        end
        else begin
            correct_count++;
        end
    endfunction
    function void reference_model(FIFO_transaction F_txn);
        if(!F_txn.rst_n) begin
            fifo_ref.delete();
            data_out_ref = 0;
        end
        else begin
            if (F_txn.wr_en && F_txn.rd_en) begin

                if(fifo_ref.size() == 0)
                    fifo_ref.push_back(F_txn.data_in);

                else if (fifo_ref.size() == FIFO_DEPTH)
                    data_out_ref = fifo_ref.pop_front();

                else begin
                    data_out_ref = fifo_ref.pop_front();
                    fifo_ref.push_back(F_txn.data_in);
                end
            end

            else if (F_txn.wr_en && !F_txn.rd_en) begin
                if (fifo_ref.size() != FIFO_DEPTH)
```

```

        fifo_ref.push_back(F_txn.data_in);
    end

    else if (!F_txn.wr_en && F_txn.rd_en) begin
        if (fifo_ref.size() != 0)
            data_out_ref = fifo_ref.pop_front();
        end
    end
    data_view = data_out_ref;
endfunction
endclass
endpackage

```

- Interface

```

interface fifo_interface ( input bit clk);

    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    logic [FIFO_WIDTH-1:0] data_in;
    logic rst_n, wr_en, rd_en;
    logic [FIFO_WIDTH-1:0] data_out;
    logic wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;

    modport DUT (input rd_en, clk, data_in, rst_n, wr_en ,
        output data_out, wr_ack, overflow, full, empty, almostfull,
almostempty, underflow);
    modport TEST (output data_in, rst_n, wr_en, rd_en,
        input full, empty, almostfull, data_out, wr_ack, overflow,
almostempty, underflow, clk);
    modport MON (input clk , data_out,data_in, rst_n, wr_en, rd_en,full, empty,
almostfull, almostempty, wr_ack, overflow, underflow);
endinterface

```

- *Transcation class*

```
package FIFO_transaction_pkg;

class FIFO_transaction;
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    rand bit rst_n, wr_en, rd_en;
    rand bit [FIFO_WIDTH-1:0] data_in;
    bit [FIFO_WIDTH-1:0] data_out;
    bit full, empty, almostfull, almostempty, underflow;
    bit wr_ack, overflow;
    integer RD_EN_ON_DIST;
    integer WR_EN_ON_DIST;

    function new(integer RD_EN_ON_DIST = 30, integer WR_EN_ON_DIST = 70);
        this.RD_EN_ON_DIST = RD_EN_ON_DIST;
        this.WR_EN_ON_DIST = WR_EN_ON_DIST;
    endfunction
// FIFO_14
    constraint reset
    {
        rst_n dist {0 := 10, 1 := 90};
    }
//FIFO_15
    constraint write_enable_cons
    {
        wr_en dist {1 := WR_EN_ON_DIST, 0 := 100-WR_EN_ON_DIST};
    }
//FIFO_16
    constraint read_enable_cons
    {
        rd_en dist {1 := RD_EN_ON_DIST, 0 := 100-RD_EN_ON_DIST};
    }
endclass

endpackage
```


- *Top*

```
module top();
    bit clk;

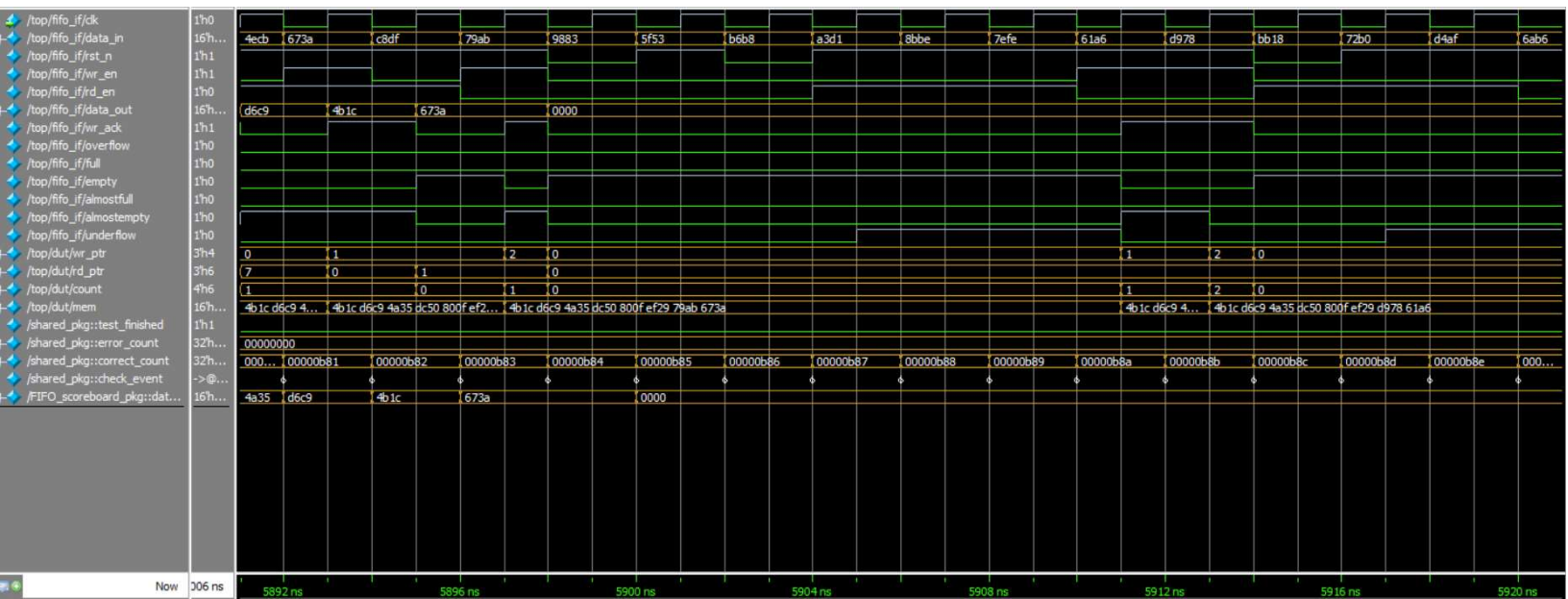
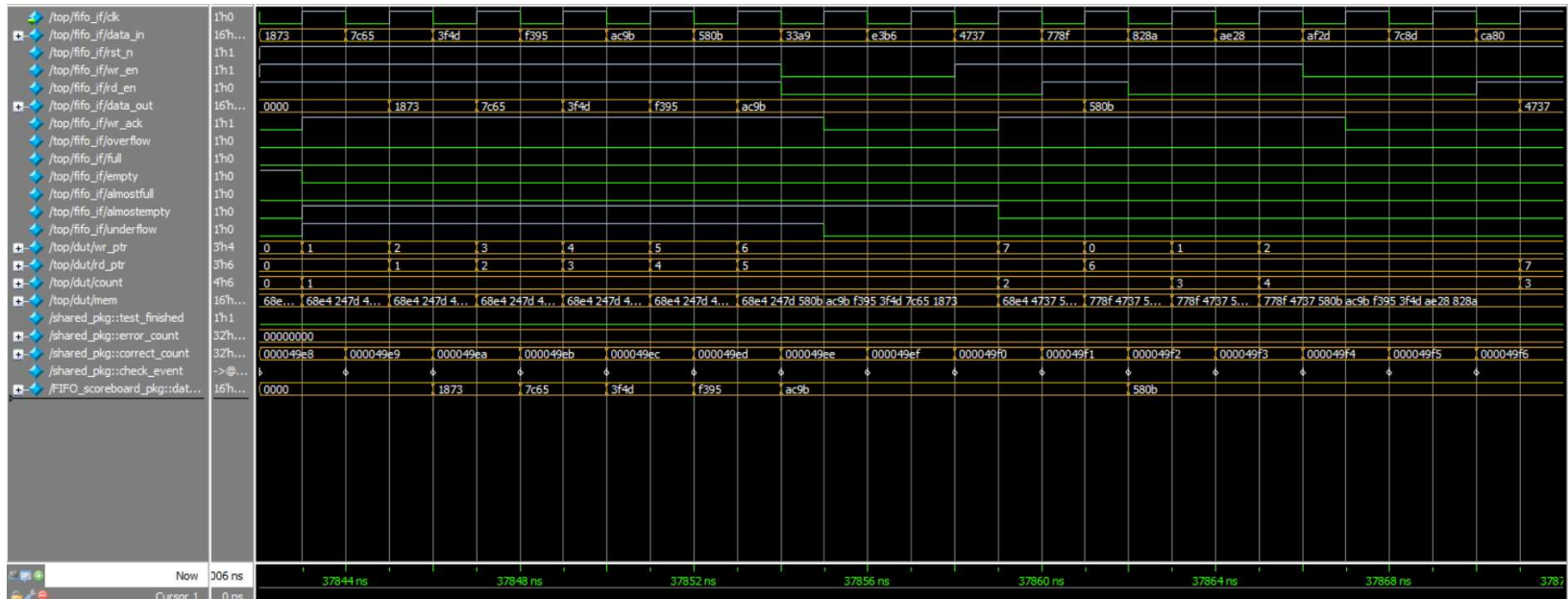
    initial begin
        clk = 0;
        forever
            #1 clk = ~clk;
    end
    fifo_interface fifo_if(clk);
    FIFO dut(fifo_if);
    fifo_tb tb(fifo_if);
    fifo_monitor mon(fifo_if);

    //FIFO_20
    always_comb begin
        if (!fifo_if.rst_n) begin

            async_reset_prop: assert final (
                (fifo_if.data_out      == '0) &&
                (fifo_if.full          == 1'b0) &&
                (fifo_if.empty         == 1'b1) &&
                (fifo_if.almostfull    == 1'b0) &&
                (fifo_if.almostempty   == 1'b0) &&
                (fifo_if.wr_ack        == 1'b0) &&
                (fifo_if.overflow       == 1'b0) &&
                (fifo_if.underflow     == 1'b0)
            ) ;
            async_reset_cover: cover final (
                (fifo_if.data_out      == '0) &&
                (fifo_if.full          == 1'b0) &&
                (fifo_if.empty         == 1'b1) &&
                (fifo_if.almostfull    == 1'b0) &&
                (fifo_if.almostempty   == 1'b0) &&
                (fifo_if.wr_ack        == 1'b0) &&
                (fifo_if.overflow       == 1'b0) &&
                (fifo_if.underflow     == 1'b0)
            );
        end
    end
endmodule
```

4- Simulation Results & Coverage

4.1 Waveform



4.2 Code Coverage Report

- Branch Coverage

```
# Branch Coverage:
#   Enabled Coverage      Bins   Hits   Misses   Coverage
#   -----
#   Branches             28     28     0   100.00%
#
# =====Branch Details=====
#
# Branch Coverage for instance /top/dut
#
#   Line   Item           Count   Source
#   ---   ---
#   File FIFO.sv
# -----IF Branch-----
#   11           21742   Count coming in to IF
#   11     1       3656       if (!fifo_if.rst_n) begin
#
#   16     1       8967       else if (fifo_if.wr_en && count < fifo_if.FIFO_DEPTH)
begin
#
#   21     1       9119       else begin
#
# Branch totals: 3 hits of 3 branches = 100.00%
#
# -----IF Branch-----
#   23           9119   Count coming in to IF
#   23     1        26       if (fifo_if.full & fifo_if.wr_en)
#
#   25     1       9093       else
#
# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
#   31           21742   Count coming in to IF
#   31     1       3656       if (!fifo_if.rst_n) begin
#
#   36     1       6280       else if (fifo_if.rd_en && count != 0) begin
#
#   41     1      11806       else begin
#
# Branch totals: 3 hits of 3 branches = 100.00%
#
# -----IF Branch-----
#   42           11806   Count coming in to IF
#   42     1       2852       if (fifo_if.empty && fifo_if.rd_en)
#
#   44     1       8954       else
#
```

```

# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
# 50          19627  Count coming in to IF
# 50          1      3612          if (!fifo_if.rst_n) begin
#
# 53          1      16015          else begin
#
# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
# 54          16015  Count coming in to IF
# 54          1      4458          if      (({fifo_if.wr_en, fifo_if.rd_en} ==
2'b10) && !fifo_if.full)
#
# 56          1      3144          else if (({fifo_if.wr_en, fifo_if.rd_en} == 2'b01)
&& !fifo_if.empty)
#
# 59          1      3786          else if ({fifo_if.wr_en, fifo_if.rd_en} == 2'b11)
begin
#
#          4627  All False Count
# Branch totals: 4 hits of 4 branches = 100.00%
#
# -----IF Branch-----
# 60          3786  Count coming in to IF
# 60          1      1384          if (fifo_if.empty)
#
#          2402  All False Count
# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
# 62          3786  Count coming in to IF
# 62          1      11          if (fifo_if.full)
#
#          3775  All False Count
# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
# 68          10303  Count coming in to IF
# 68          1      31          assign fifo_if.full = (count == fifo_if.FIFO_DEPTH)? 1 : 0;
#
# 68          2      10272          assign fifo_if.full = (count == fifo_if.FIFO_DEPTH)? 1 : 0;
#
# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
# 69          10303  Count coming in to IF
# 69          1      2782          assign fifo_if.empty = (count == 0)? 1 : 0;
#
# 69          2      7521          assign fifo_if.empty = (count == 0)? 1 : 0;
#
# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
# 70          10303  Count coming in to IF

```

```

# 70      1          72      assign fifo_if.almostfull = (count == fifo_if.FIFO_DEPTH-1)? 1 : 0;
// bug fixed
#
# 70      2          10231     assign fifo_if.almostfull = (count == fifo_if.FIFO_DEPTH-1)? 1 : 0;
// bug fixed
#
# Branch totals: 2 hits of 2 branches = 100.00%
#
# -----IF Branch-----
# 71      1          10303     Count coming in to IF
# 71      1          3578      assign fifo_if.almostempty = (count == 1)? 1 : 0;
#
# 71      2          6725      assign fifo_if.almostempty = (count == 1)? 1 : 0;
#
# Branch totals: 2 hits of 2 branches = 100.00%
#

```

- Statement Coverage

```

# Statement Coverage:
# Enabled Coverage      Bins  Hits  Misses Coverage
# -----
# Statements           28    28    0  100.00%
#
# =====Statement Details=====
#
# Statement Coverage for instance /top/dut --
#
# Line   Item          Count  Source
# ----   ---
# File FIFO.sv
# 1              module FIFO(fifo_interface.DUT fifo_if);
# 2
# 3              localparam max_fifo_addr = $clog2(fifo_if.FIFO_DEPTH);
# 4
# 5              reg [fifo_if.FIFO_WIDTH-1:0] mem [fifo_if.FIFO_DEPTH-1:0];
# 6
# 7              reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
# 8              reg [max_fifo_addr:0] count;
# 9
# 10         1          21742      always @(posedge fifo_if.clk or negedge fifo_if.rst_n) begin
# 11         1              if (!fifo_if.rst_n) begin
# 12         1          3656          wr_ptr <= 0;
# 13         1          3656          fifo_if.wr_ack <= 0; // bug fixed
# 14         1          3656          fifo_if.overflow <= 0; // bug fixed
# 15         1              end
# 16         1              else if (fifo_if.wr_en && count < fifo_if.FIFO_DEPTH) begin
# 17         1          8967          mem[wr_ptr] <= fifo_if.data_in;
# 18         1          8967          fifo_if.wr_ack <= 1;
# 19         1          8967          wr_ptr <= wr_ptr + 1;

```

```

# 20      end
# 21      else begin
# 22          1          9119          fifo_if.wr_ack <= 0;
# 23          if (fifo_if.full & fifo_if.wr_en)
# 24          1          26          fifo_if.overflow <= 1;
# 25          else
# 26          1          9093          fifo_if.overflow <= 0;
# 27      end
# 28      end
# 29
# 30          1          21742      always @(posedge fifo_if.clk or negedge fifo_if.rst_n) begin
# 31          if (!fifo_if.rst_n) begin
# 32          1          3656          rd_ptr <= 0;
# 33          1          3656          fifo_if.underflow <= 0; // bug fixed
# 34          1          3656          fifo_if.data_out <= 0; // bug fixed
# 35          end
# 36          else if (fifo_if.rd_en && count != 0) begin
# 37          1          6280          fifo_if.data_out <= mem[rd_ptr];
# 38          1          6280          rd_ptr <= rd_ptr + 1;
# 39          end
# 40          // bug fixed
# 41          else begin
# 42          if (fifo_if.empty && fifo_if.rd_en)
# 43          1          2852          fifo_if.underflow <= 1;
# 44          else
# 45          1          8954          fifo_if.underflow <= 0;
# 46          end
# 47      end
# 48
# 49          1          19627      always @(posedge fifo_if.clk or negedge fifo_if.rst_n) begin
# 50          if (!fifo_if.rst_n) begin
# 51          1          3612          count <= 0;
# 52          end
# 53          else begin
# 54          if ( ({fifo_if.wr_en, fifo_if.rd_en} == 2'b10) &&
!fifo_if.full)
# 55          1          4458          count <= count + 1;
# 56          else if ( ({fifo_if.wr_en, fifo_if.rd_en} == 2'b01) &&
!fifo_if.empty)
# 57          1          3144          count <= count - 1;
# 58          // bug fixed
# 59          else if ( {fifo_if.wr_en, fifo_if.rd_en} == 2'b11) begin
# 60          if (fifo_if.empty)
# 61          1          1384          count <= count + 1;
# 62          if (fifo_if.full)
# 63          1          11          count <= count - 1;
# 64          end
# 65      end
# 66      end
# 67
# 68          1          10304      assign fifo_if.full = (count == fifo_if.FIFO_DEPTH)? 1 : 0;
# 69          1          10304      assign fifo_if.empty = (count == 0)? 1 : 0;
# 70          1          10304      assign fifo_if.almostfull = (count == fifo_if.FIFO_DEPTH-1)? 1 : 0; //
bug fixed
# 71          1          10304      assign fifo_if.almostempty = (count == 1)? 1 : 0;

```

- Toggle Coverage

```
# =====Toggle
Details=====
#
# Toggle Coverage for instance /top/fifo_if --
#
#           Node   1H->0L   0L->1H "Coverage"
#           -----
#           almostempty      1      1   100.00
#           almostfull      1      1   100.00
#           clk             1      1   100.00
#           data_in[15-0]    1      1   100.00
#           data_out[15-0]   1      1   100.00
#           empty           1      1   100.00
#           full            1      1   100.00
#           overflow        1      1   100.00
#           rd_en           1      1   100.00
#           rst_n           1      1   100.00
#           underflow       1      1   100.00
#           wr_ack          1      1   100.00
#           wr_en           1      1   100.00
#
# Total Node Count   =    43
# Toggled Node Count =    43
# Untoggled Node Count =    0
#
# Toggle Coverage    = 100.00% (86 of 86 bins)
```

4.3 Functional Coverage Report

```
# Covergroup Coverage:
#   Covergroups          1   na   na 100.00%
#   Coverpoints/Crosses  16   na   na  na
#   Covergroup Bins      25   25   0 100.00%
# -----
# Covergroup              Metric    Goal    Bins  Status
# -----
# TYPE /FIFO coverage_pack/FIFO coverage/CovGroup 100.00%    100    - Covered
# covered/total bins:          25    25    -
# missing/total bins:          0    25    -
# % Hit:                        100.00%  100    -
# Coverpoint write_en_cp       100.00%    100    - Covered
# covered/total bins:          2    2    -
# missing/total bins:          0    2    -
# % Hit:                        100.00%  100    -
# bin wr_en_on                 9966    1    - Covered
# bin wr_en_off                10036    1    - Covered
# Coverpoint read_en_cp        100.00%    100    - Covered
# covered/total bins:          2    2    -
# missing/total bins:          0    2    -
# % Hit:                        100.00%  100    -
# bin rd_en_on                 10080    1    - Covered
# bin rd_en_off                9922    1    - Covered
# Coverpoint full_cp           100.00%    100    - Covered
# covered/total bins:          2    2    -
# missing/total bins:          0    2    -
# % Hit:                        100.00%  100    -
# bin full_high                64    1    - Covered
# bin full_low                 19938    1    - Covered
# Coverpoint almostfull_cp     100.00%    100    - Covered
# covered/total bins:          2    2    -
# missing/total bins:          0    2    -
# % Hit:                        100.00%  100    -
# bin almostfull_high          129    1    - Covered
# bin almostfull_low           19873    1    - Covered
# Coverpoint empty_cp          100.00%    100    - Covered
# covered/total bins:          2    2    -
# missing/total bins:          0    2    -
# % Hit:                        100.00%  100    -
# bin empty_high               6265    1    - Covered
# bin empty_low                13737    1    - Covered
# Coverpoint almostempty_cp    100.00%    100    - Covered
# covered/total bins:          2    2    -
# missing/total bins:          0    2    -
# % Hit:                        100.00%  100    -
# bin almostempty_high         6636    1    - Covered
# bin almostempty_low          13366    1    - Covered
# Coverpoint overflow_cp       100.00%    100    - Covered
# covered/total bins:          2    2    -
# missing/total bins:          0    2    -
# % Hit:                        100.00%  100    -
# bin overflow_high            31    1    - Covered
```



```

#   bin overflow_low          19971      1      -   Covered
#   Coverpoint underflow_cp    100.00%    100      -   Covered
#   covered/total bins:       2      2      -
#   missing/total bins:       0      2      -
#   % Hit:                     100.00%    100      -
#   bin underflow_high        3705      1      -   Covered
#   bin underflow_low        16297      1      -   Covered
#   Coverpoint wr_ack_cp      100.00%    100      -   Covered
#   covered/total bins:       2      2      -
#   missing/total bins:       0      2      -
#   % Hit:                     100.00%    100      -
#   bin wr_ack_high           8967      1      -   Covered
#   bin wr_ack_low           11035      1      -   Covered
#   Cross wr_rd_full          100.00%    100      -   Covered
#   covered/total bins:       1      1      -
#   missing/total bins:       0      1      -
#   % Hit:                     100.00%    100      -
#   Auto, Default and User Defined Bins:
#   bin read_write_full       20002      1      -   Covered
#   Cross wr_rd_almostfull    100.00%    100      -   Covered
#   covered/total bins:       1      1      -
#   missing/total bins:       0      1      -
#   % Hit:                     100.00%    100      -
#   Auto, Default and User Defined Bins:
#   bin read_write_almostfull 20002      1      -   Covered
#   Cross wr_rd_empty         100.00%    100      -   Covered
#   covered/total bins:       1      1      -
#   missing/total bins:       0      1      -
#   % Hit:                     100.00%    100      -
#   Auto, Default and User Defined Bins:
#   bin read_write_empty      20002      1      -   Covered
#   Cross wr_rd_almostempty   100.00%    100      -   Covered
#   covered/total bins:       1      1      -
#   missing/total bins:       0      1      -
#   % Hit:                     100.00%    100      -
#   Auto, Default and User Defined Bins:
#   bin read_write_almostempty 20002      1      -   Covered
#   Cross wr_rd_overflow      100.00%    100      -   Covered
#   covered/total bins:       1      1      -
#   missing/total bins:       0      1      -
#   % Hit:                     100.00%    100      -
#   Auto, Default and User Defined Bins:
#   bin read_write_overflow   20002      1      -   Covered
#   Cross wr_rd_underflow     100.00%    100      -   Covered
#   covered/total bins:       1      1      -
#   missing/total bins:       0      1      -
#   % Hit:                     100.00%    100      -
#   Auto, Default and User Defined Bins:
#   bin read_write_underflow  20002      1      -   Covered
#   Cross wr_rd_wrack         100.00%    100      -   Covered
#   covered/total bins:       1      1      -
#   missing/total bins:       0      1      -
#   % Hit:                     100.00%    100      -
#   Auto, Default and User Defined Bins:
#   bin read_write_ack        20002      1      -   Covered

```

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage
/FIFO_coverage_pack/FIFO_coverage		100.00%						
TTYPE CovGroup		100.00%	100	100.00...		✓	auto(1)	
CVP CovGroup::write_en_cp		100.00%	100	100.00...		✓		
bin wr_en_on		9966	1	100.00...		✓		
bin wr_en_off		10036	1	100.00...		✓		
CVP CovGroup::read_en_cp		100.00%	100	100.00...		✓		
bin rd_en_on		10080	1	100.00...		✓		
bin rd_en_off		9922	1	100.00...		✓		
CVP CovGroup::full_cp		100.00%	100	100.00...		✓		
bin full_high		64	1	100.00...		✓		
bin full_low		19938	1	100.00...		✓		
CVP CovGroup::almostfull_cp		100.00%	100	100.00...		✓		
bin almostfull_high		129	1	100.00...		✓		
bin almostfull_low		19873	1	100.00...		✓		
CVP CovGroup::empty_cp		100.00%	100	100.00...		✓		
bin empty_high		6265	1	100.00...		✓		
bin empty_low		13737	1	100.00...		✓		
CVP CovGroup::almostempty_cp		100.00%	100	100.00...		✓		
bin almostempty_high		6636	1	100.00...		✓		
bin almostempty_low		13366	1	100.00...		✓		
CVP CovGroup::overflow_cp		100.00%	100	100.00...		✓		
bin overflow_high		31	1	100.00...		✓		
bin overflow_low		19971	1	100.00...		✓		
CVP CovGroup::underflow_cp		100.00%	100	100.00...		✓		
bin underflow_high		3705	1	100.00...		✓		
bin underflow_low		16297	1	100.00...		✓		
CVP CovGroup::wr_ack_cp		100.00%	100	100.00...		✓		
bin wr_ack_high		8967	1	100.00...		✓		
bin wr_ack_low		11035	1	100.00...		✓		
CROSS CovGroup::wr_rd_full		100.00%	100	100.00...		✓		
bin read_write_full		20002	1	100.00...		✓		
CROSS CovGroup::wr_rd_almostfull		100.00%	100	100.00...		✓		
bin read_write_almostfull		20002	1	100.00...		✓		
CROSS CovGroup::wr_rd_empty		100.00%	100	100.00...		✓		

bin read_write_empty		20002	1	100.00...		✓		
CROSS CovGroup::wr_rd_almostempty		100.00%	100	100.00...		✓		
bin read_write_almostempty		20002	1	100.00...		✓		
CROSS CovGroup::wr_rd_overflow		100.00%	100	100.00...		✓		
bin read_write_overflow		20002	1	100.00...		✓		
CROSS CovGroup::wr_rd_underflow		100.00%	100	100.00...		✓		
bin read_write_underflow		20002	1	100.00...		✓		
CROSS CovGroup::wr_rd_wrack		100.00%	100	100.00...		✓		
bin read_write_ack		20002	1	100.00...		✓		

```

# Simulation done at time          40006
# Correct Count: 20002 , Error Count: 0
# ** Note: $stop      : fifo_monitor.sv(46)
#   Time: 40006 ns  Iteration: 1  Instance: /top/mon

```

4.4 Assertions Report

# Assertion Coverage:								
#	Assertions	13	13	0	100.00%			
#	-----							
#	Name	File(Line)	Failure	Pass	Vacuous	Disable	Attempt	Active Peak Active
ATV								
#		Count	Count	Count	Count	Count	Count	Count
#	-----							
#	/top/dut/assert_count_threshold_prop							
#	FIFO.sv(193)	0	18086	0	1917	20003	0	1 off
#	/top/dut/assert_rd_ptr_threshold_prop							
#	FIFO.sv(189)	0	18086	0	1917	20003	0	1 off
#	/top/dut/assert_wr_ptr_threshold_prop							
#	FIFO.sv(185)	0	18086	0	1917	20003	0	1 off
#	/top/dut/assert_rd_ptr_wraparound_prop							
#	FIFO.sv(181)	0	273	17777	1953	20003	0	1 off
#	/top/dut/assert_wr_ptr_wraparound_prop							
#	FIFO.sv(178)	0	476	17556	1971	20003	0	1 off
#	/top/dut/assert_almostempty_prop							
#	FIFO.sv(175)	0	6015	12071	1917	20003	0	1 off
#	/top/dut/assert_almostfull_prop							
#	FIFO.sv(172)	0	116	17970	1917	20003	0	1 off
#	/top/dut/assert_full_flag_prop							
#	FIFO.sv(169)	0	58	18028	1917	20003	0	1 off
#	/top/dut/assert_empty_flag_prop							
#	FIFO.sv(166)	0	5655	12431	1917	20003	0	1 off
#	/top/dut/assert_underflow_prop							
#	FIFO.sv(163)	0	2588	15234	2181	20003	0	2 off
#	/top/dut/assert_overflow_prop							
#	FIFO.sv(160)	0	24	18060	1919	20003	0	2 off
#	/top/dut/assert_wr_ack_prop							
#	FIFO.sv(157)	0	8131	9119	2752	20003	1	2 off
#	/top/dut/assert_reset_prop							
#	FIFO.sv(154)	0	1917	18086	0	20003	0	2 off

Assertions					
Name	Assertion Type	Language	Enable	Failure Count	Pass Count
/top/async_reset_prop	Immediate	SVA	on	0	1739
/top/dut/assert_reset_prop	Concurrent	SVA	on	0	1917
/top/dut/assert_wr_ack_prop	Concurrent	SVA	on	0	8131
/top/dut/assert_overflow_prop	Concurrent	SVA	on	0	24
/top/dut/assert_underflow_prop	Concurrent	SVA	on	0	2588
/top/dut/assert_empty_flag_prop	Concurrent	SVA	on	0	5655
/top/dut/assert_full_flag_prop	Concurrent	SVA	on	0	58
/top/dut/assert_almostfull_prop	Concurrent	SVA	on	0	116
/top/dut/assert_almostempty_prop	Concurrent	SVA	on	0	6015
/top/dut/assert_wr_ptr_wraparound_prop	Concurrent	SVA	on	0	476
/top/dut/assert_rd_ptr_wraparound_prop	Concurrent	SVA	on	0	273
/top/dut/assert_wr_ptr_threshold_prop	Concurrent	SVA	on	0	18086
/top/dut/assert_rd_ptr_threshold_prop	Concurrent	SVA	on	0	18086
/top/dut/assert_count_threshold_prop	Concurrent	SVA	on	0	18086
/top/tb/#ublk#217929410#22/immed__23	Immediate	SVA	on	0	20000